Assignment 1: Build the Model from Scratch

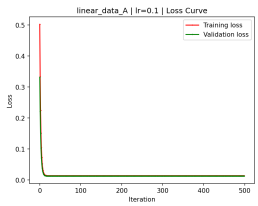
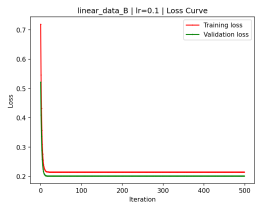
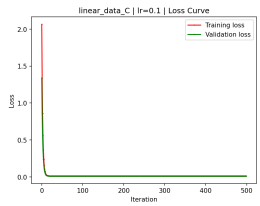
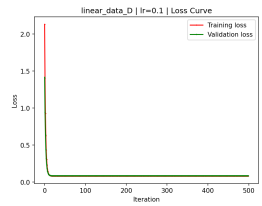
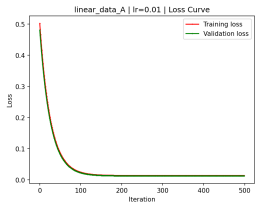
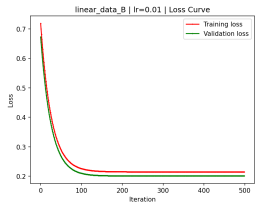
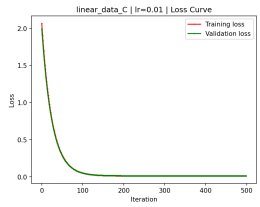
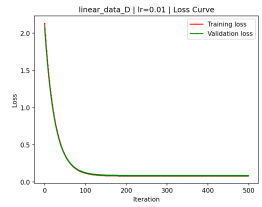
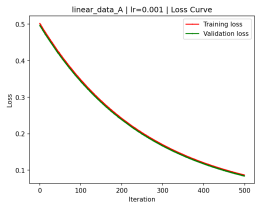
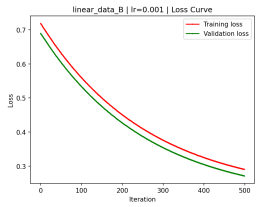
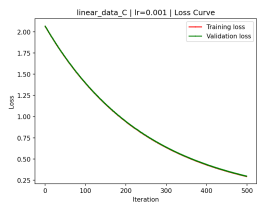
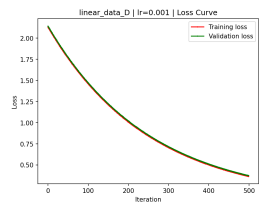
Student ID: 314540066

GitHub Link: (https://github.com/Ashurali/DM2026-Assignment-1-MKS)

---

1. Linear Regression (20 points)

1(a) Loss Curves

	Dataset A	Dataset B	Dataset C	Dataset D
LR = 0.1				
LR = 0.01				
LR = 0.001				

1(b) Evaluation

Dataset	Learning Rate	MSE	MAE	RMSE	R-squared
A	0.1	0.0134	0.1010	0.1156	0.5690
	0.01	0.0134	0.1010	0.1156	0.5690
	0.001	0.0832	0.2369	0.2884	-1.6836
B	0.1	0.2137	0.4038	0.4623	0.0448
	0.01	0.2137	0.4038	0.4623	0.0448
	0.001	0.2687	0.4404	0.5184	-0.2011
C	0.1	0.0132	0.0987	0.1148	0.9954
	0.01	0.0132	0.0987	0.1148	0.9954
	0.001	0.2852	0.4273	0.5341	0.8994
D	0.1	0.0824	0.2467	0.2870	0.9716
	0.01	0.0824	0.2467	0.2870	0.9716
	0.001	0.3540	0.4778	0.5950	0.8780

1(c) Observations

Effect of Learning Rate on Convergence:

- **LR = 0.1** converges almost immediately (within the first 50 iterations) across all datasets. The loss curve drops sharply and flattens, indicating the model quickly reaches its optimal weights.
- **LR = 0.01** converges more gradually (around 100–200 iterations), producing a smoother loss curve. It reaches the same final performance as LR = 0.1, showing that both learning rates are sufficient for 500 iterations.
- **LR = 0.001** fails to converge within 500 iterations across all datasets. The loss curve is still clearly decreasing at iteration 500. This is reflected in degraded evaluation metrics:
  - For datasets A and B, R-squared becomes **negative** (-1.6836 and -0.2011), indicating the model performs worse than simply predicting the mean of the data.
  - For datasets C and D, R-squared remains positive (0.8994 and 0.8780) but is noticeably lower than the converged values (0.9954 and 0.9716). The stronger underlying linear signal in these datasets means even partial convergence captures much of the variance.

Differences Across Datasets:

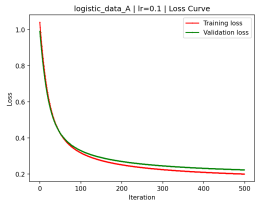
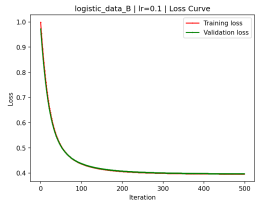
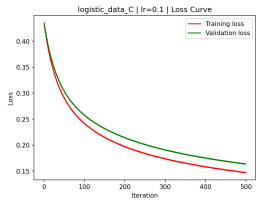
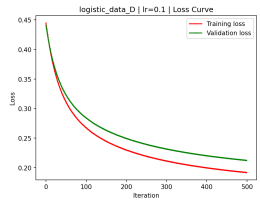
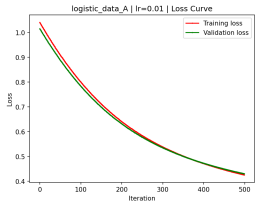
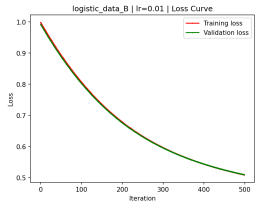
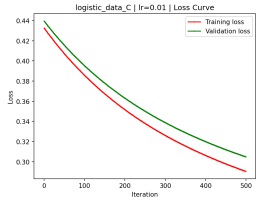
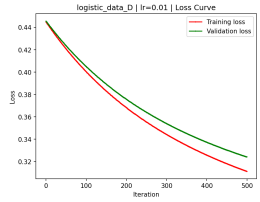
- **Dataset B** has the lowest R-squared even when fully converged (0.0448), indicating the data contains substantial noise relative to the linear signal.
- **Datasets C and D** (5 features, 8000 samples) achieve the best performance when converged, with R-squared values of 0.9954 and 0.9716 respectively.
- **Dataset A** falls in between (R-squared = 0.5690), with a moderate linear signal in 1D data.

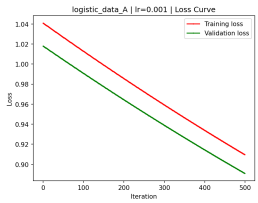
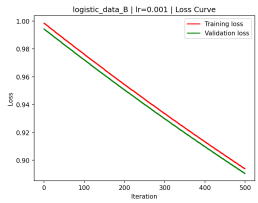
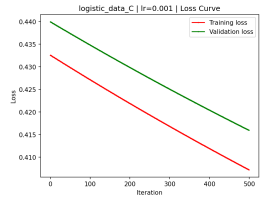
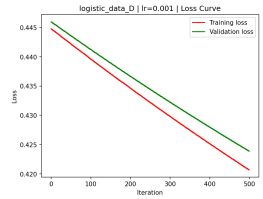
Key Takeaway: A learning rate that is too small requires more iterations to converge. With a fixed iteration budget (500), LR = 0.001 is insufficient. However, LR = 0.1 is not harmful here – it converges quickly and achieves the same result as 0.01.

---

2. Logistic Regression (20 points)

2(a) Loss Curves (n_iteration = 500)

	Dataset A	Dataset B	Dataset C	Dataset D
LR = 0.1				
LR = 0.01				

	Dataset A	Dataset B	Dataset C	Dataset D
LR = 0.001				

2(b) Evaluation (n_iteration = 500)

Dataset	LR	Accuracy	Precision	Recall	F1-score
A	0.1	0.9050	0.9087	0.9171	0.9128
	0.01	0.8775	0.8784	0.8986	0.8884
	0.001	0.3875	0.4364	0.4424	0.4394
B	0.1	0.7800	0.7793	0.8160	0.7972
	0.01	0.7625	0.7600	0.8066	0.7826
	0.001	0.3925	0.4286	0.4387	0.4336
C	0.1	0.9762	0.9852	0.9719	0.9785
	0.01	0.9456	0.9707	0.9303	0.9500
	0.001	0.8406	0.8774	0.8290	0.8525
D	0.1	0.9225	0.9330	0.9266	0.9298
	0.01	0.9113	0.9346	0.9029	0.9185
	0.001	0.8319	0.8668	0.8228	0.8442

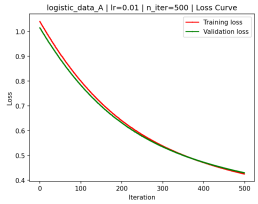
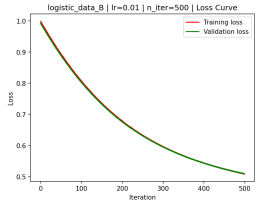
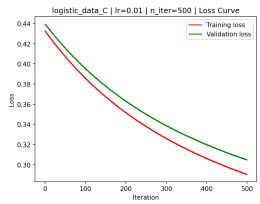
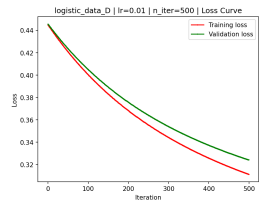
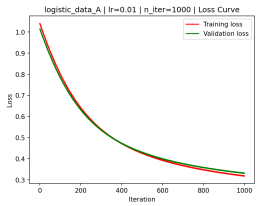
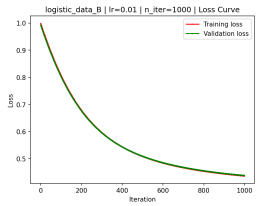
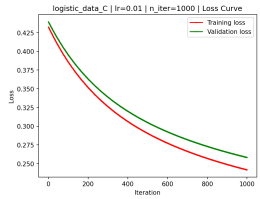
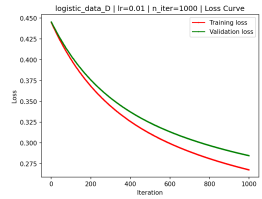
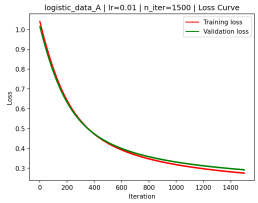
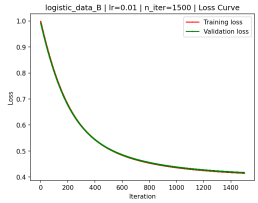
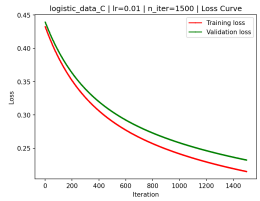
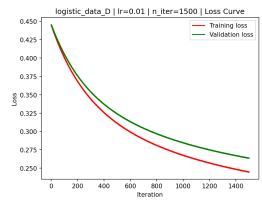
2(c) Observations

The general trend follows linear regression: higher learning rates converge faster, and LR = 0.001 fails to converge within 500 iterations. However, there are notable differences:

- **LR = 0.1** achieves the best metrics across all datasets. Unlike linear regression where LR = 0.1 and 0.01 reached identical results, here LR = 0.1 consistently outperforms LR = 0.01. This is because even LR = 0.01 has **not fully converged** at 500 iterations – its loss curve is still visibly decreasing with a steep slope at iteration 500.
- **LR = 0.01** shows degraded but still reasonable performance (e.g., Data A: 0.8775 accuracy vs 0.9050 at LR = 0.1).
- **LR = 0.001** essentially fails – accuracy drops to near-random levels (~0.39 for A and B). The model has learned almost nothing.

Comparison with Linear Regression: Logistic regression converges more slowly than linear regression for the same learning rate. This is because the sigmoid activation compresses outputs, effectively reducing the effective gradient magnitude. The same LR = 0.01 that was sufficient for linear regression in 500 iterations is insufficient for logistic regression.

2(d) Loss Curves — Varying n_iteration (lr = 0.01)

	Dataset A	Dataset B	Dataset C	Dataset D
n_iter = 500				
n_iter = 1000				
n_iter = 1500				

2(e) Evaluation — Varying n_iteration (lr = 0.01)

Dataset	n_iter	Accuracy	Precision	Recall	F1-score
A	500	0.8775	0.8784	0.8986	0.8884
	1000	0.9025	0.9045	0.9171	0.9108
	1500	0.9025	0.9083	0.9124	0.9103
B	500	0.7625	0.7600	0.8066	0.7826
	1000	0.7775	0.7783	0.8113	0.7945
	1500	0.7775	0.7783	0.8113	0.7945
C	500	0.9456	0.9707	0.9303	0.9500
	1000	0.9744	0.9885	0.9651	0.9767
	1500	0.9788	0.9897	0.9719	0.9807
D	500	0.9113	0.9346	0.9029	0.9185
	1000	0.9263	0.9394	0.9266	0.9330
	1500	0.9256	0.9393	0.9255	0.9323

2(f) Observations

Increasing the number of iterations with a fixed LR = 0.01 confirms that this learning rate had not yet converged at 500 iterations:

- **500 to 1000 iterations** yields a clear improvement across all datasets. For example, Dataset A jumps from 0.8775 to 0.9025 accuracy, nearly matching the LR = 0.1 result (0.9050). This confirms LR = 0.01 was not a bad learning rate – it simply needed more iterations.
- **1000 to 1500 iterations** shows diminishing returns. Datasets A, B, and D show no further improvement – the accuracy and all metrics remain essentially identical. This indicates the model has reached convergence around 1000 iterations for these datasets.

- **Dataset C** is the exception, showing continued marginal improvement from 1000 to 1500 (accuracy 0.9744 to 0.9788), suggesting its loss surface requires slightly more iterations to fully converge due to higher dimensionality.

These results directly confirm that the performance gap between LR = 0.1 and LR = 0.01 observed in 2(a)-2(c) was purely a convergence issue, not a fundamental problem with the learning rate.

---

3. Data Preprocessing (20 points)

3(a) Median and Standard Deviation (Before Imputation)

The following 5 columns contain missing values:

Column	Missing Count	Median	Std Dev
SepalLengthCm	28	6.3000	1.0371
SepalWidthCm	57	2.9000	0.3896
PetalLengthCm	150	5.0856	1.5828
PetalWidthCm	22	1.6000	0.7067
BranchLength	24	16.3000	1.0352

Note: PetalLengthCm has the most missing values (150 out of 500 = 30%).

3(b) After Nearest Neighbors Imputation (KNN, k=5)

Column	Median (Before)	Median (After)	Std (Before)	Std (After)
SepalLengthCm	6.3000	6.3000	1.0371	1.0093
SepalWidthCm	2.9000	2.9000	0.3896	0.3724
PetalLengthCm	5.0856	5.0357	1.5828	1.5150
PetalWidthCm	1.6000	1.7000	0.7067	0.6946
BranchLength	16.3000	16.3000	1.0352	1.0110

Observation

KNN imputation replaces missing values with the weighted average of the K nearest neighbors' values. Compared to simpler methods (like mean/median imputation):

- **Median** values stay approximately the same or shift slightly, since KNN imputes values based on similar samples rather than the global average. The imputed values are context-aware and tend to be close to what the actual values would have been.
- **Standard deviation decreases slightly** after imputation. This is because KNN imputation fills in values that are averages of neighbors, which are inherently less extreme than actual observed values. This averaging effect reduces the spread of the distribution.
- The effect is most pronounced for **PetalLengthCm** (30% missing), where the imputation of 150 values has the largest impact on the distribution (std drops from 1.5828 to 1.5150).
- Columns with fewer missing values (PetalWidthCm: 22, BranchLength: 24) show minimal changes since fewer values are being imputed.

4. Data Exploration (20 points)

4(a) Histogram of PetalWidthCm

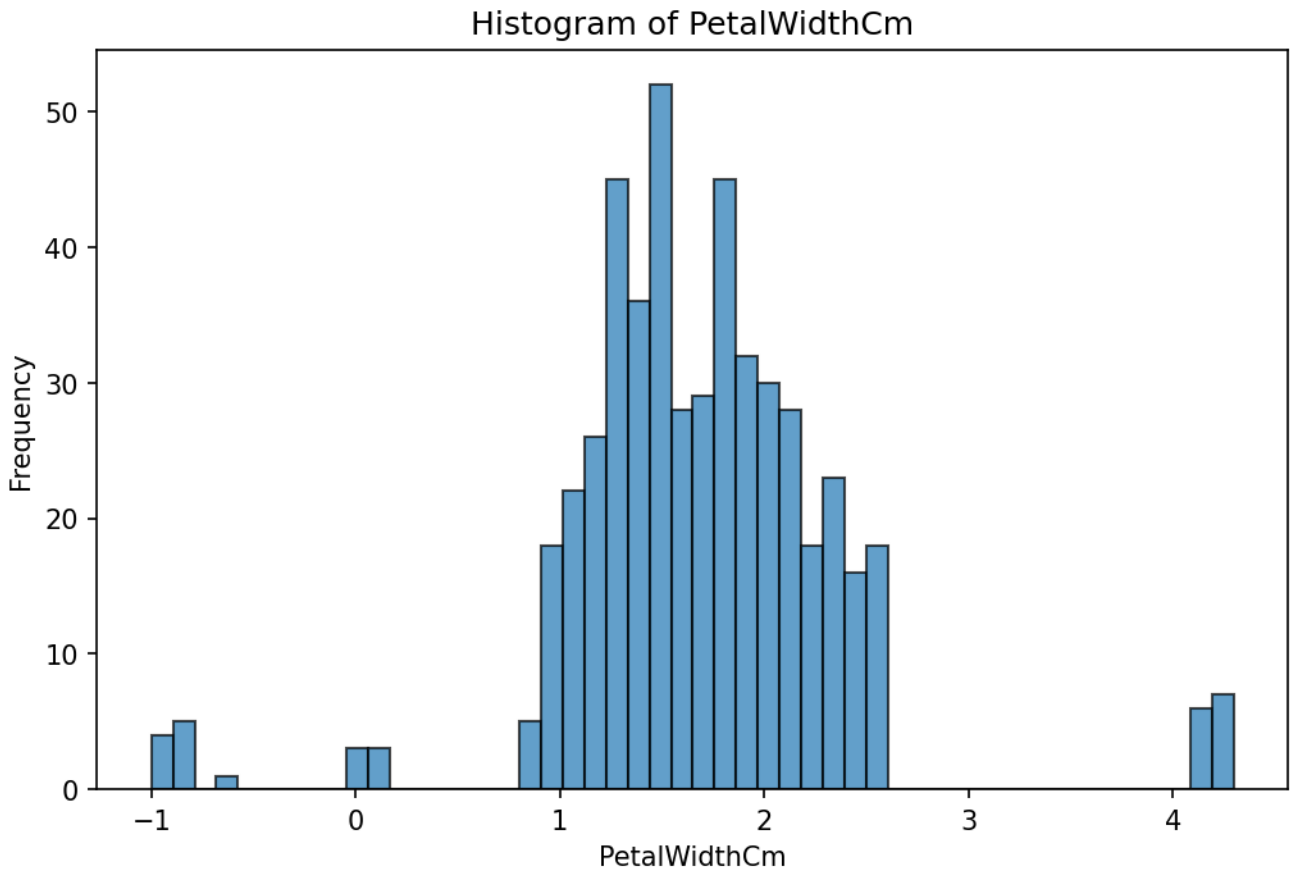


Figure 1: Histogram of PetalWidthCm

The histogram of PetalWidthCm shows an approximately normal distribution cluster centered around 1.0–2.5 cm. However, there are notable outliers:

- **Negative values (-1 to -0.5):** About 10 data points with physically impossible negative petal widths, indicating data entry errors or intentionally injected noise in the dataset.
- **Extreme high values (4.0–4.3):** A small cluster of unusually large values far from the main distribution.
- The core distribution (1.0–2.5) is roughly bell-shaped with a peak around 1.4–1.5 cm.

The existence of two peaks in the histogram shows a slight bimodal tendency in the data. This indicates the data contains two species — Iris-versicolor and Iris-virginica — as seen in the CSV data. This likely means one species tends towards narrower petals while the other towards wider petals.

4(b) Feature with Largest Positive Correlation

PetalWidthCompactness has the largest positive Pearson correlation with PetalWidthCm: **r = 0.9917**.

This near-perfect correlation indicates PetalWidthCompactness is essentially a direct derivative of PetalWidthCm.

4(c) Top 5 Features with Strongest Negative Correlations

Rank	Feature	Pearson r
1	SepalWidthMajorAxis	-0.0964
2	SepalGlossIndex	-0.0952
3	SepalWidthCompactness	-0.0885
4	SepalWidthCurvature	-0.0813
5	SepalWidthMinorAxis	-0.0744

All negative correlations are very weak ($|r| < 0.1$). Notably, 4 of the 5 features are SepalWidth-derived features that are highly correlated with each other (inter-correlation $r > 0.94$), indicating they are essentially redundant representations of the same underlying SepalWidth signal.

4(d) Boxplot

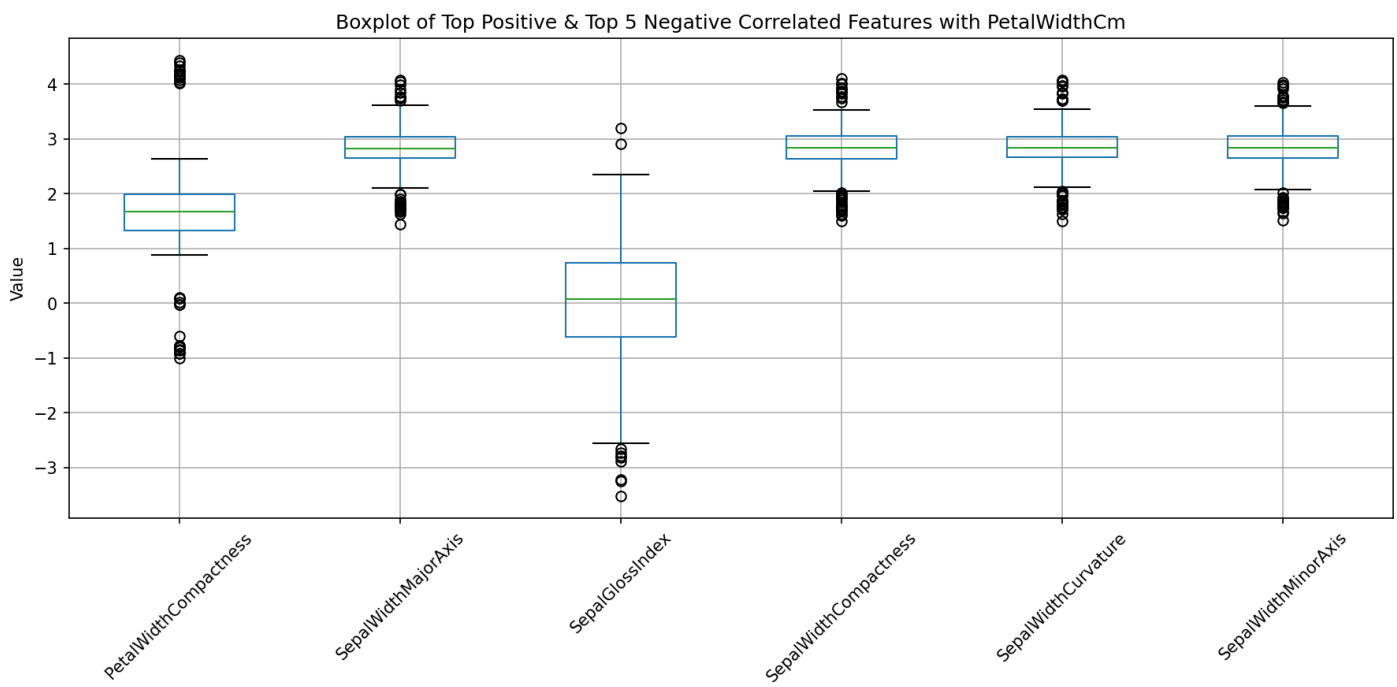


Figure 2: Boxplot

The boxplot reveals:

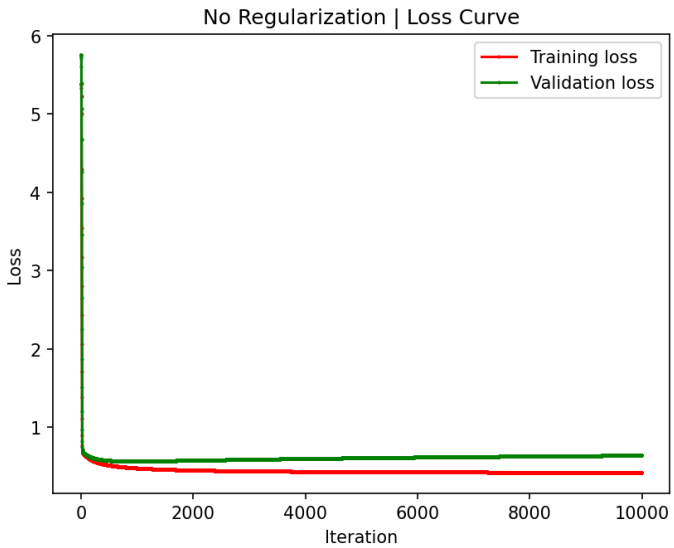
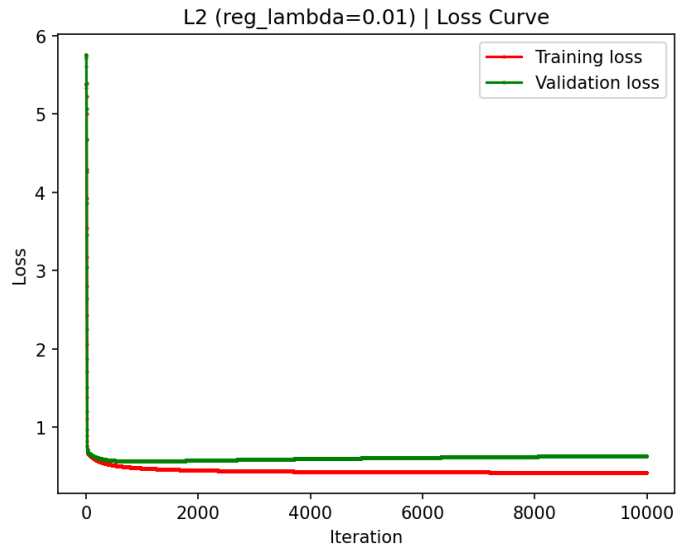
- **PetalWidthCompactness**: Distribution mirrors PetalWidthCm (median ~1.7, IQR ~1.3–2.0), with outliers below 0 and above 4 – consistent with its near-perfect correlation ($r = 0.9917$).
- **SepalWidthMajorAxis, SepalWidthCompactness, SepalWidthCurvature, SepalWidthMinorAxis**: These four features have nearly identical boxplots (median ~2.84, IQR ~2.6–3.1), visually confirming their high inter-correlation. They represent redundant SepalWidth-derived measurements.

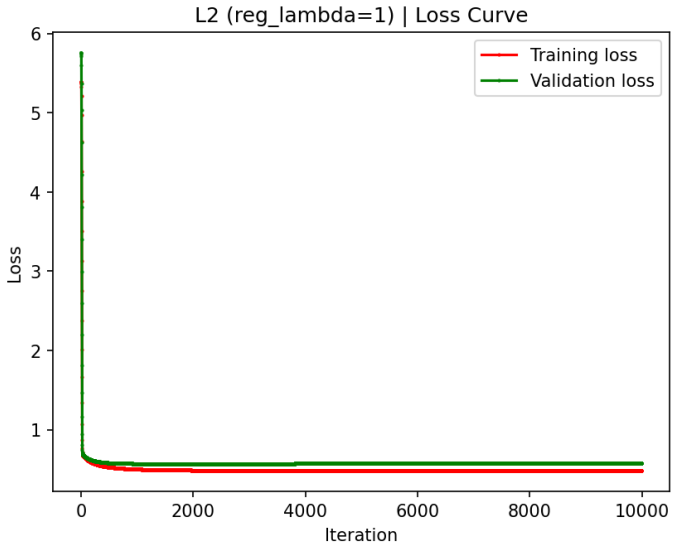
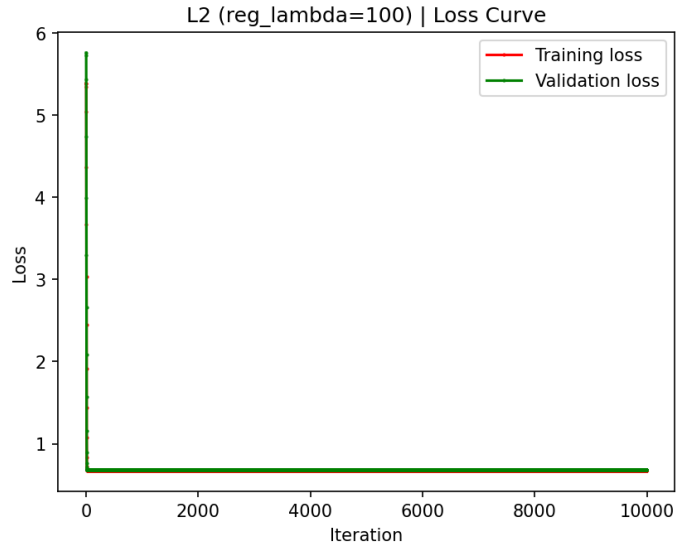
- **SepalGlossIndex**: The only feature centered near 0 with a wider spread (IQR ~-0.6 to 0.7), indicating it is a standardized/different type of feature compared to the SepalWidth group.

---

5. Regularization (20 points)

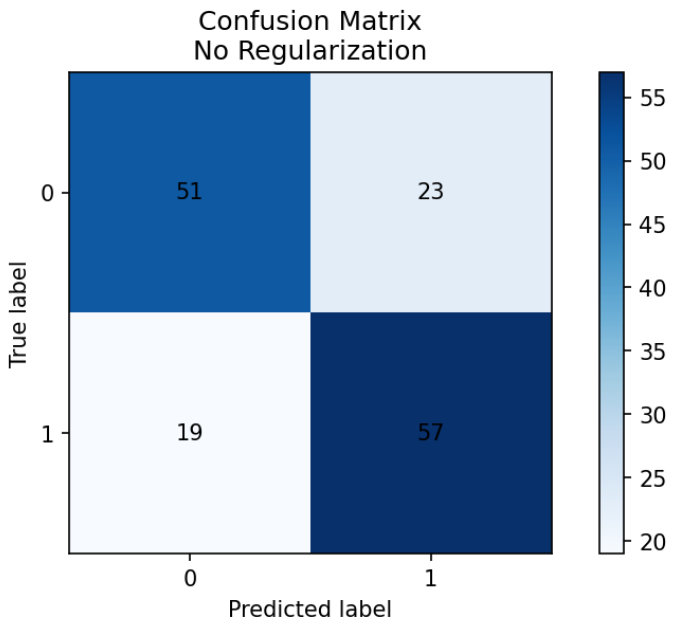
5(a) Loss Curves

No Regularization	L2 (reg_lambda=0.01)
	

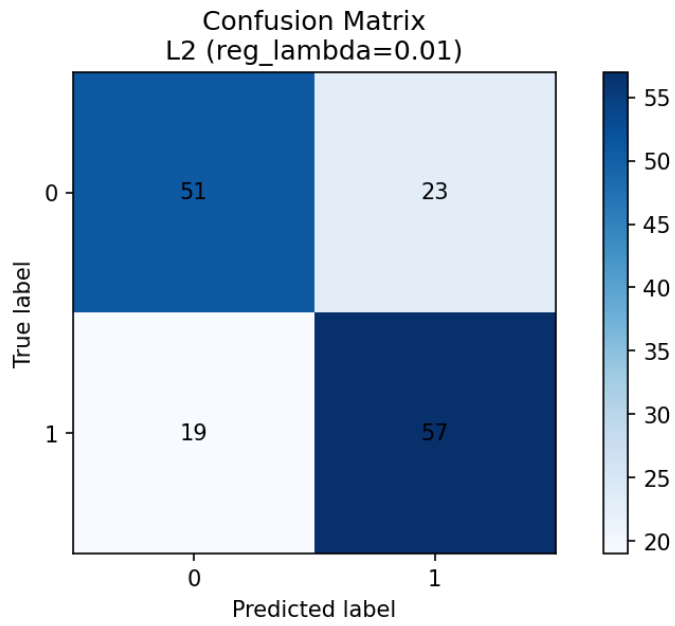
L2 (reg_lambda=1)	L2 (reg_lambda=100)
	

5(b) Evaluation

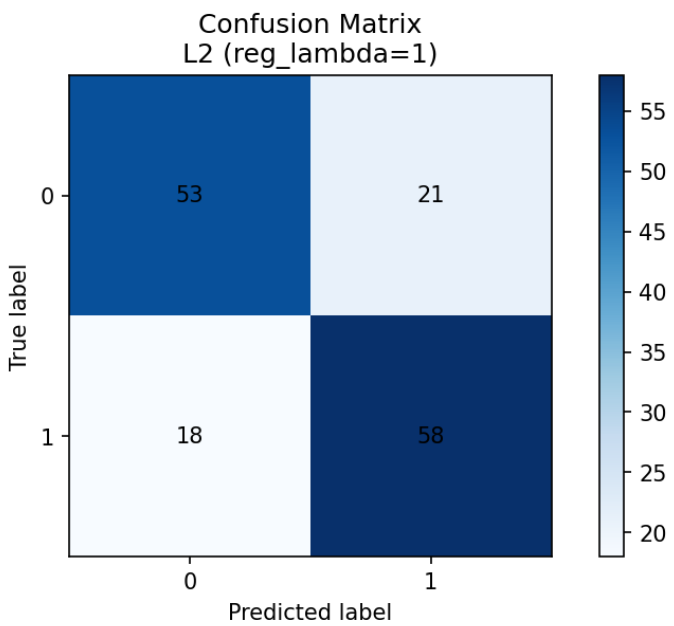
No Regularization



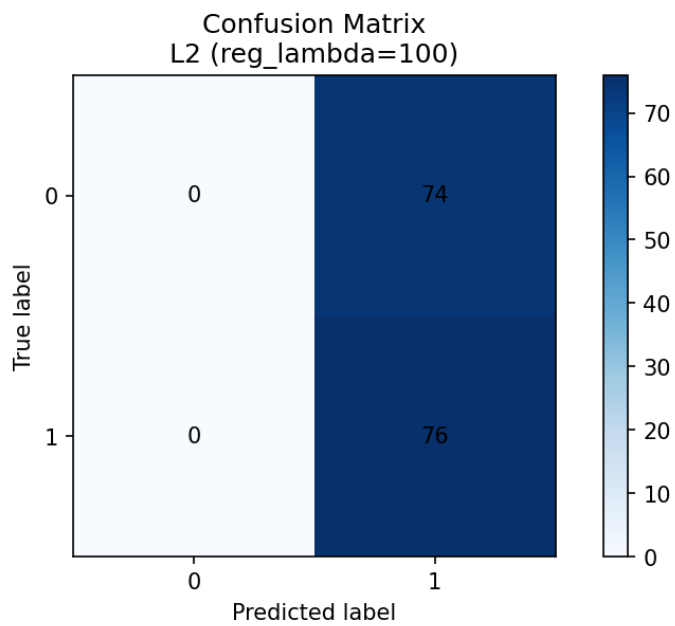
L2 (reg_lambda=0.01)



L2 (reg_lambda=1)



L2 (reg_lambda=100)



Setting	Accuracy	Precision	Recall	F1-score
No Regularization	0.7200	0.7125	0.7500	0.7308
L2 (lambda=0.01)	0.7200	0.7125	0.7500	0.7308
L2 (lambda=1)	0.7400	0.7342	0.7632	0.7484
L2 (lambda=100)	0.5067	0.5067	1.0000	0.6726

5(c) Observations

L2 regularization adds a penalty proportional to the squared magnitude of weights, encouraging smaller weight values. The results show a clear spectrum:

- **lambda=0.01**: No meaningful effect. The metrics are identical to no regularization. The penalty is too weak to influence the optimization.
- **lambda=1**: The sweet spot. Accuracy improves from 0.7200 to 0.7400, with gains across all metrics (Precision: 0.7125 to 0.7342, F1: 0.7308 to 0.7484). The model has been effectively regularized – large, potentially overfitting weights have been shrunk. This suggests the unregularized model was overfitting to the training data with its 70 features.
- **lambda=100**: Over-regularization. The weights are almost entirely suppressed, meaning the model has lost its ability to discriminate. It predicts the positive class for nearly every sample (Recall = 1.0000), but Precision drops to 0.5067 (barely above random) and Accuracy falls to 0.5067. The model behaves like a constant predictor.

Key Takeaway: Regularization strength must be carefully tuned. Too weak (0.01) has no effect; too strong (100) destroys the model's discriminative power by suppressing all weights to near-zero. The optimal value (lambda=1) reduces overfitting by constraining the weight magnitudes while preserving the model's ability to learn meaningful patterns from the 70-feature dataset.